

Crafty Controller Wake Proxy

Smart 'Wake-on-Ping' TCP proxy microservice for Minecraft
Resource optimization for 24/7 hosts and NAS servers

Document Details

Project:	Crafty Controller Wake Proxy
Language:	Python 3.11+ / Dockerized
License:	MIT (Open Source)
Compatible Panel:	Crafty Controller v4
Generation Date:	August 2026
Project Author:	Jonatan / Open Source Community

Introduction

This manual documents the operation, configuration, and deployment of the Crafty Controller Wake Proxy microservice. This system allows you to drastically optimize system resources (CPU and RAM) by automatically shutting down inactive Minecraft servers and starting them on-demand in a transparent manner as soon as a player tries to connect. The microservice is fully dockerized and designed to run robustly and unattended on any Linux, Windows, or NAS (such as Synology DSM) host.

Table of Contents

1. Features and Detailed Operation	Page 2
2. Advanced Security and Protection Systems	Page 3
3. Network Scheme and Ports	Page 4
4. Obtaining Credentials in Crafty 4	Page 5
5. Full Source Code (Dockerfile, docker-compose.yml)	Page 6
6. Compose File and Deployment (Linux and Synology NAS)	Page 7
7. Management and Best Practices	Page 8
8. MIT License	Page 8

1. Features and Detailed Operation

The microservice is a transparent TCP reverse proxy positioned between the Minecraft client and the actual game server managed by Crafty Controller v4.

a) Wake-on-Ping with Handshake Filter:

The proxy listens on the designated external port. When a client attempts to connect, the proxy reads the first bytes and parses the Minecraft Handshake protocol. If it detects a simple server list ping (`next_state = 1`) or a port scan, it closes the socket immediately and does not boot the server. If the player actively clicks 'Join Server' (`next_state = 2`), the proxy intercepts the connection, queries the Crafty Controller API (`GET /api/v2/servers/{serverID}/stats`), and if offline, performs an authorized HTTP POST request with a JWT Token to start the server.

The proxy retains the player's connection and initiates an asynchronous checking loop, querying Crafty's status every 2 seconds. As soon as the server is confirmed ONLINE, it links network sockets, sends the initial handshake packet, and forwards all gameplay traffic in real time.

b) Auto-Sleep with Double-Layered Verification:

To optimize hardware resources on the host, a background thread monitors the network socket. To avoid false positives, server list auto-pings and port scans do not reset the inactivity timer, which starts counting only when the last real player leaves.

Furthermore, the proxy continuously monitors the server state in Crafty and, as soon as it detects the server transitioning from OFFLINE to ONLINE (whether started manually from the Crafty panel or automatically by the proxy), it resets the inactivity timer immediately. This guarantees a full grace period equal to `IDLE_TIMEOUT_SECONDS` from the moment the server boots, before the proxy can consider shutting it down. Once the idle threshold expires, the proxy queries the Crafty REST API for the actual online player count. The server will only shut down if there are 0 active TCP connections AND the API confirms that the online player count is 0.

2. Advanced Security and Protection Systems

The microservice incorporates specific protection mechanisms designed to guarantee energy efficiency and host optimization, preventing accidental boot-ups and involuntary shutdowns:

a) Smart Handshake Filter (Minecraft Handshake Parser):

This filter analyzes the first bytes of all incoming TCP connections to the proxy's public port, decoding the Minecraft communication protocol in real time:

- If a Status Ping (`next_state = 1`) is detected, the client is only querying status (e.g., refreshing the multiplayer server list or scanning ports). If the server is offline, the proxy discards and closes the connection immediately, preventing accidental wake-ups.
- If a Login Request (`next_state = 2`) is detected, this indicates the player clicked 'Join Server'. Only in this case does the proxy send the start command to the Crafty API.

b) Double-Layered Shutdown Verification (Inactivity Double Check):

This system acts as a double-confirmation shield before shutting down the server instance:

- Manual start respected: The proxy continuously monitors the Crafty server state. The moment it detects the server transition from OFFLINE to ONLINE (whether started manually from the Crafty panel or by the proxy), it resets the inactivity timer automatically. This ensures a full grace period of `IDLE_TIMEOUT_SECONDS` (configured in `.env`) is always observed from the moment the server boots, preventing it from being shut down immediately after a manual start.
- When the timer expires with 0 active TCP connections, the proxy queries the Crafty API (`GET /api/v2/servers/{serverID}/stats`) and verifies the actual online player count ('online'). The server will only shut down if the proxy registers 0 TCP connections AND the API confirms the count is 0.
- If active players are reported by the API, the inactivity timer is reset, preventing accidental shutdowns.

3. Network Scheme and Ports

- Public / Proxy Port (LISTEN_PORT):

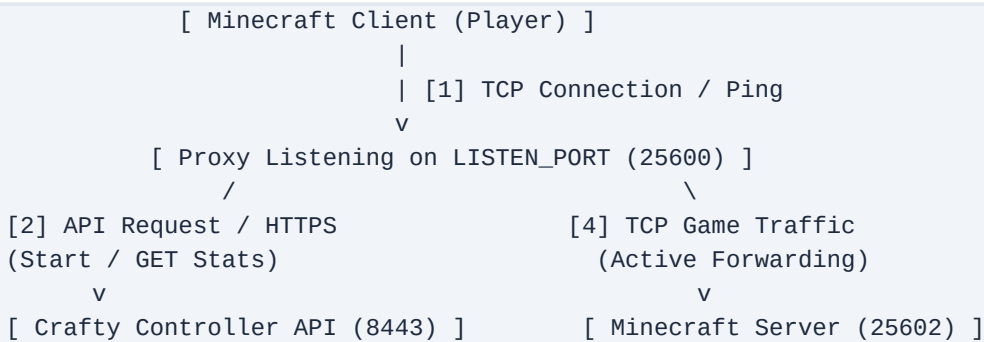
The port exposed to the internet on your router (Port Forwarding). By default, this is 25565 or 25600. It is the only port remote players need to know.

- Internal Minecraft Port (INTERNAL_PORT):

The port that Crafty assigns to the Minecraft instance (e.g., 25602). This port must remain hidden and firewalled from the internet, accessible only locally by the TCP proxy.

- Crafty URL (CRAFTY_URL):

The HTTPS web administration panel on the default port 8443. The docker container needs local subnet access to query and start/stop the instance. The proxy ignores self-signed certificate alerts via software warning disabling.



4. Obtaining Credentials in Crafty 4

A) Generate Crafty API Token (API_TOKEN)

1. Log in to the Crafty Controller v4 web panel as an Administrator.
2. Click on your user icon (top right corner) and select 'Account Settings'.
3. Go to the 'API Keys' section.
4. In the 'Name - What would you like to call this API Token?' field, enter a name (e.g., 'WakeProxy').
5. Under the permissions list (COMMANDS, TERMINAL, LOGS, etc.), check the 'Full Access' checkbox.
6. Click the green '+Create' button.
7. Once created, click on the 'Get a token' button in the upper table.
8. IMPORTANT: Copy the token displayed. Do not click this button again. If you click it a second time, Crafty will generate a new token and invalidate the one you copied previously.

B) Identify the Server UUID (SERVER_ID)

1. In the main Crafty menu, go to the 'Servers' tab.
2. Click on the specific Minecraft server you want to manage with the proxy.
3. Look at your browser's URL address bar.
4. Copy the complete alphanumeric UUID that appears after the '?id=' parameter.

For example, in the URL:

`https://panel.crafty.local:8443/panel/server_detail?id=516a6a24-9b36-419b-a010-85f8c6b753a9`

The UUID to copy is: 516a6a24-9b36-419b-a010-85f8c6b753a9

5. Full Source Code

Environment Configuration File (.env)

Modify the variables in the .env file (created from .env.example). IMPORTANT: DO NOT modify the source code file src/proxy.py, all configurations must be done exclusively in the .env file:

```
CRAFTY_URL=https://192.168.1.100:8443
API_TOKEN=your_crafty_api_jwt_token_here
SERVER_ID=your_minecraft_server_uuid_here
TARGET_HOST=127.0.0.1
INTERNAL_PORT=25602
LISTEN_PORT=25600
IDLE_TIMEOUT_SECONDS=600
```

Dockerfile

Docker container environment based on Python slim:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src/proxy.py .
CMD ["python", "-u", "proxy.py"]
```

6. Compose File and Deployment

docker-compose.yml

```
version: '3.8'

services:
  crafty_controller-wake_proxy:
    build: .
    container_name: crafty_controller-wake_proxy
    restart: unless-stopped
    network_mode: bridge
    ports:
      - "${LISTEN_PORT:-25600}:${LISTEN_PORT:-25600}"
    env_file:
      - .env
    volumes:
      - ./src/proxy.py:/app/proxy.py
      - .env:/app/.env
```

Installation Guides

Linux CLI Method:

1. Create working folder: `mkdir -p "/opt/Crafty Controller-Wake Proxy" && cd "/opt/Crafty Controller-Wake Proxy"`
2. Create the `src/` folder and place `proxy.py` inside. Create `Dockerfile`, `docker-compose.yml`, `requirements.txt`, `.gitignore`, and `.env` (copied from `.env.example`).
3. Configure your actual credentials inside the `.env` file.
4. Run: `docker compose up -d --build`

Synology NAS DSM Method:

1. Create a shared folder in File Station (e.g., `/volume1/docker/Crafty Controller-Wake Proxy`) and upload the project files (`src` folder, `Dockerfile`, etc.).
2. Create a `.env` file in that same folder with your local credentials.
3. Open Container Manager, go to Projects and click Create.
4. Select the shared folder, choose 'Create docker-compose.yml', and paste the compose file.
5. Complete the wizard to build and start the container. The compose will automatically load the `.env` and map `./src/proxy.py`.

7. Management and Best Practices

Administration Commands

- View container logs in real time: `docker logs -f crafty_controller-wake_proxy`
- Restart container to apply proxy.py changes without rebuild: `docker restart crafty_controller-wake_proxy`
- Temporarily stop microservice: `docker compose down`
- Rebuild dependencies or rebuild image: `docker compose up -d --build`

Best Practices and Tips

- Inactivity Timeout: Keep `IDLE_TIMEOUT_SECONDS` between 600 (10 mins) and 1800 (30 mins). Avoid values below 5 mins to prevent shutdowns during brief player lag/disconnects.
- Security: Keep the API Token protected. Do not upload it to public repositories. We recommend using environment variables or dotenv files on shared or public production hosts.
- Routing: Ensure the Docker container has network permissions to contact both the internal game IP/port and the HTTPS port of the Crafty Controller API.

8. MIT License

Copyright (c) 2026 Jonatan

This code and documentation is distributed under the terms of the MIT License. Use, modification, distribution, and commercial exploitation are permitted without restrictions, under the sole condition of including the original copyright notice in derivative works.